

UF2

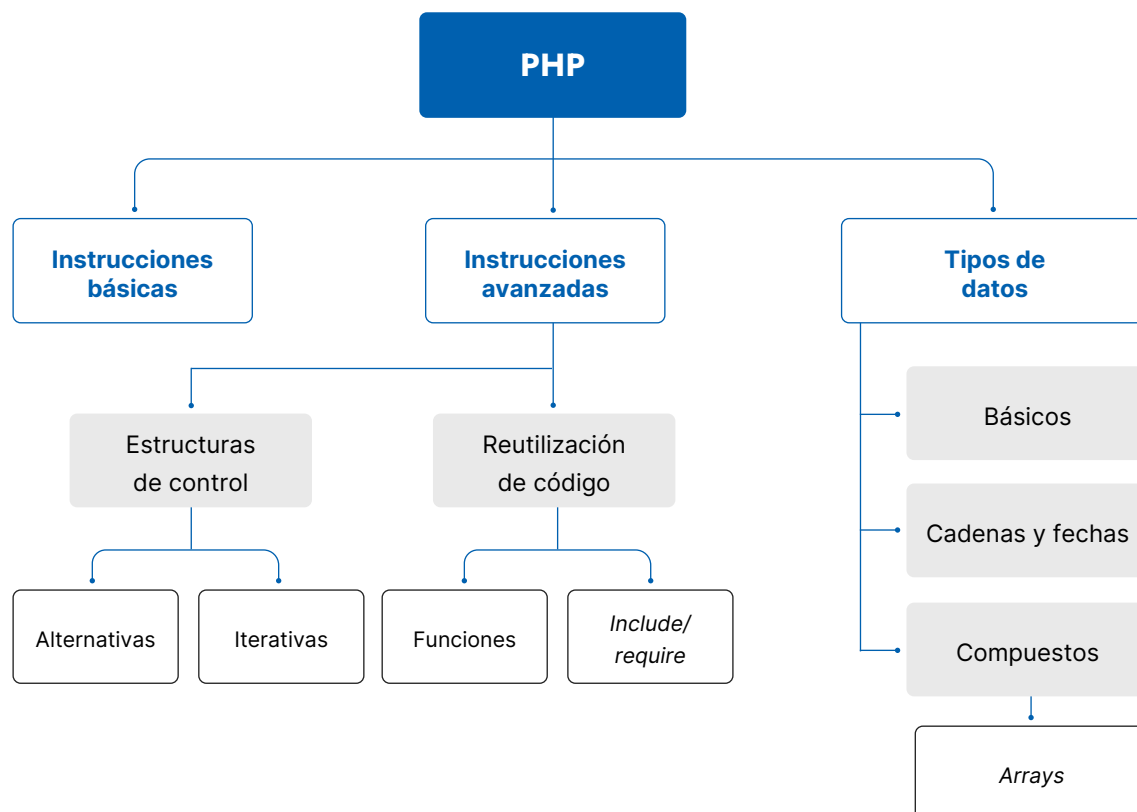
PHP Básico

Desarrollo web en entorno servidor

ÍNDICE

Mapa conceptual.....	03
1. Introducción al lenguaje PHP	04
1.1. PHP embebido en un lenguaje de marcas	04
1.2. PHP: sintaxis y operaciones básicas	05
1.3. PHP: tipos de datos simples	08
1.4. PHP: constantes, constantes predefinidas y constantes mágicas.....	09
1.5. PHP: variables con nombres generados dinámicamente.....	10
2. Cadenas y fechas	12
2.1. Cadenas	12
2.2. Fechas	15
3. Estructuras de control	18
3.1. Estructuras alternativas/condicionales: <i>if</i>	19
3.2. Estructuras alternativas/condicionales: <i>switch</i>	21
3.3. Estructuras iterativas/repetitivas: <i>while</i>	22
3.4. Estructuras iterativas/repetitivas: <i>for</i>	23
4. Arrays y otros tipos compuestos de datos	26
4.1. Arrays	26
4.2. Arrays asociativos	27
4.3. Funciones de <i>arrays</i>	29
4.4. Arrays multidimensionales.....	30
5. Funciones. Include y require	34
5.1. Funciones en PHP	34
5.2. <i>Include</i>	40
5.3. <i>Require</i>	41
5.4. <i>Include_once</i> y <i>require_once</i>	41

Mapa conceptual



01 Introducción al lenguaje PHP

Los documentos creados con el lenguaje PHP son de texto plano, con lo que, en principio, podrían crearse y modificarse con cualquier editor de textos. Sin embargo, es siempre conveniente usar un IDE (entorno integrado de desarrollo) que acepte PHP directamente a través de plugins, puesto que nos servirá para detectar errores, anticipar instrucciones y, en general, desarrollar de una manera más rápida y eficiente. Nosotros hemos decidido usar Visual Studio Code como entorno de desarrollo, Apache como servidor web para alojar las pruebas que vayamos haciendo, y un navegador para visualizar el resultado. Asumimos también que las pruebas y los ejemplos están alojados en una carpeta con el número de la unidad dentro de la carpeta raíz del servidor web, al que accederemos mediante `http://localhost`.

EJEMPLO

Puedes encontrar todo el código descrito en este apartado descargando el archivo `DWES_U02_A01.zip`, en la sección Ejemplos para trabajar y analizar.

1.1. PHP embebido en un lenguaje de marcas

Un documento PHP tiene como extensión `.php`. Aunque el documento contenga principalmente código HTML, si incluye elementos del lenguaje PHP debe obligatoriamente tener dicha extensión. Además, la parte correspondiente al código PHP se incluirá dentro de la etiqueta `<?php` (apertura) y `?>` (cierre). Esta opción de combinar en el mismo documento código cliente (generalmente un lenguaje de marcas) con código servidor se conoce como **código embebido**. En este caso, es PHP el que queda embebido dentro de un documento con HTML/CSS/JavaScript. Por ejemplo:

```
<!doctype html>
<html>
<head>
<meta charset="UTF-8">
<title>Ejemplo 01</title>
</head>
<body>
<h1>Mi primera página PHP</h1>
```

```
<?php
//código php
?>
</p>
</body>
</html>
```

Si tenemos el servidor Apache en funcionamiento, el documento en la carpeta **ud02** dentro de la raíz, y su nombre es **ejemplo01.php**, podemos visualizar el código cliente que se genera abriendo en el navegador la URL **http://localhost/ud02/ejemplo01.php**

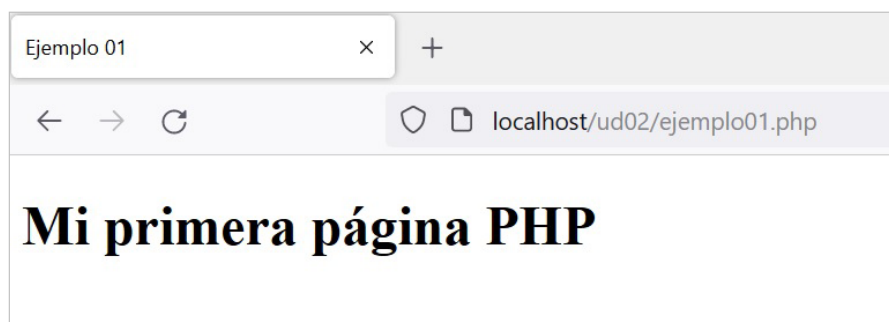


Figura 1.

Lógicamente, solo vemos el resultado del código HTML, porque no hemos incluido todavía nada de PHP. Veamos su sintaxis y operaciones básicas para poder crear nuestros primeros ejemplos.

1.2. PHP: sintaxis y operaciones básicas

En primer lugar, veamos cómo escribir comentarios en una aplicación PHP:

```
//comentario en una línea
# comentario en una línea
/* Comentario
en
varias
líneas */
```

Algunas características importantes de PHP:

- » No exige que se especifique el tipo de las variables; se deduce por su contenido y su uso.
- » El nombre de las variables empieza por el signo **\$**. Puede incluir números y guiones bajos, pero no empezar con un número ni contener espacios o caracteres especiales distintos del guion bajo. Las palabras reservadas no pueden usarse en los nombres de las variables.

- » Las cadenas pueden delimitarse con comillas dobles ("cadena") o simples ('cadena').
- » Las instrucciones acaban con un punto y coma.
- » PHP distingue entre mayúsculas y minúsculas (lenguaje *case-sensitive*).

Operadores principales:

Aritméticos	De comparación	Lógicos
+ Suma - Resta * Multiplicación / División % Módulo (resto) ** Potencia (elevado a)	== Igual a != o <> Distinto de === Idéntico a (en valor y tipo) !== No idéntico a >, >= Mayor que, mayor o igual que <, <= Menor que, menor o igual que	&& y (<i>and</i>). Devuelve <i>true</i> si todas las expresiones evaluadas son verdaderas o (<i>or</i>). Devuelve <i>false</i> si todas las expresiones evaluadas son falsas. ! no (<i>not</i>). Devuelve <i>true</i> si la expresión evaluada es falsa, y <i>false</i> si es verdadera.

Esquema 1.

```
$a=2;
$b=3;
$c=$a**$b; // se guarda en c el valor de a elevado a b
```

El signo igual (=) se usa para asignar un valor a una variable. Recordemos que las expresiones literales se encierran entre comillas dobles o simples, aunque más adelante veremos que hay una diferencia importante entre ambas opciones.

```
$a=1;
$b="Hola";
```

Se pueden usar también los operadores combinados de incremento/decremento y asignación para asignar a una variable el resultado de realizar una operación sobre la propia variable.

```
$a++; // es equivalente a $a = $a + 1
$a--; // es equivalente a $a = $a - 1
$a+=2; // es equivalente a $a = $a + 2
$a*=5; // es equivalente a $a = $a * 5
```

Para concatenar cadenas de caracteres podemos usar el operador punto (.), así como el operador combinado (.=), para asignarle a una variable el resultado de concatenarle otra cadena. Para mostrar por pantalla el valor de una variable o una expresión, disponemos de la instrucción **echo**.

```
$cadena1="Hola"; // asignamos "Hola" a la cadena1
$cadena2=","; // asignamos una coma a la cadena2
$cadena3="mundo"; // asignamos "mundo" a la cadena3
// guardamos en la cadena 1 el resultado de concatenarle la cadena2
$cadena1.=$cadena2;
// mostramos la cadena1 concatenada con un espacio en blanco y la cadena3
echo $cadena1." ".$cadena3;
```

La instrucción `echo` permite combinar variables y expresiones de diferentes tipos. También podemos incluir código HTML en el contenido que mostramos por pantalla.

```
<?php
$numero=1;
$cadena="Hola";
echo "El valor de número es $numero y el de cadena es $cadena";
echo "El valor de número es ".$numero." y el de cadena es ".$cadena;
echo "<br/>"; // un salto de línea
echo "El valor de número es ".$numero." <br/> y el de cadena es ".$cadena;
echo "<h3>El valor de número es $numero y el de cadena es $cadena</h3>";
?>
```

El resultado por pantalla del código anterior sería:

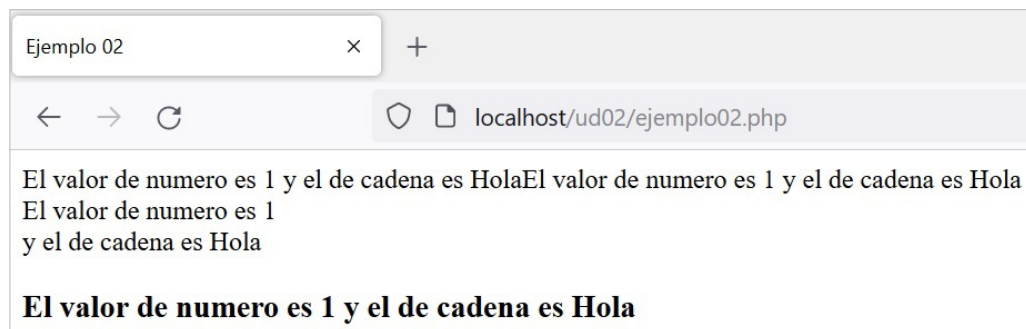


Figura 2.

Como se ve, la instrucción `echo` por sí misma no produce un salto de línea. En este caso, usamos la etiqueta HTML `<br/>` para provocarlo. También se ve en la figura cómo combinamos cadenas literales, expresiones PHP y código HTML para mostrar por pantalla texto con formato. Finalmente, vemos también que las variables PHP pueden incluirse en el texto literal, o concatenarse a él mediante el operador punto. Ahora bien, si el texto va entre comillas simples, entonces no se mostrará el valor de la variable, sino su nombre.

El resultado de:

```
$nombre="Elena";
echo "Hola, $nombre. ¿Cómo estás hoy?<br/>";
echo 'Hola, $nombre. ¿Cómo estás hoy?<br/>';
```

Sería:

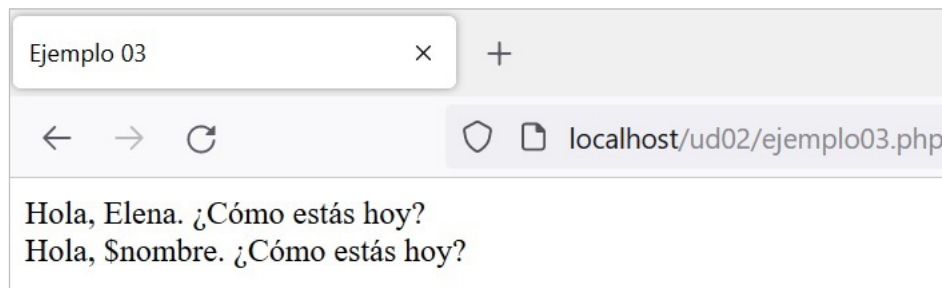
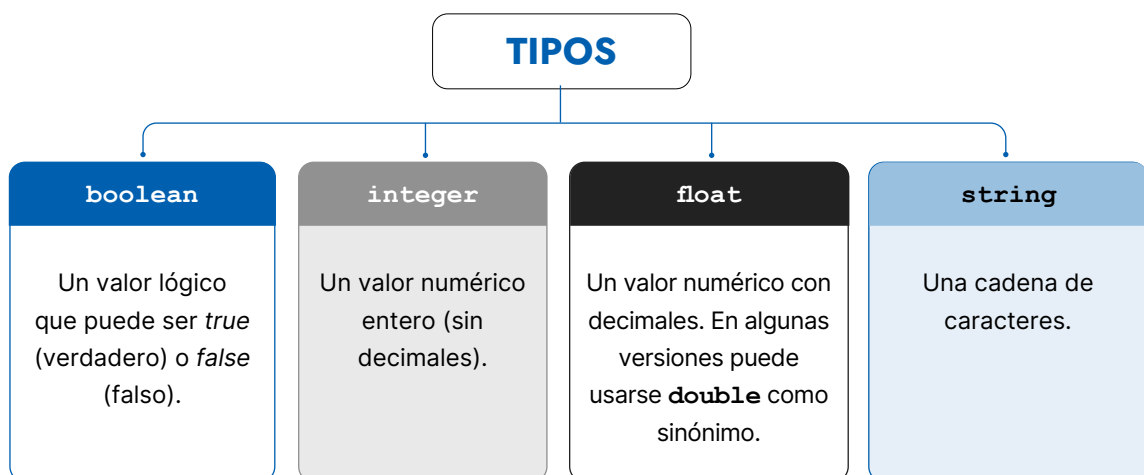


Figura 3.

1.3. PHP: tipos de datos simples

Aunque hemos comentado que no es obligatorio especificar el tipo de datos de una variable, sí que existen (y pueden usarse en determinados casos, veremos más adelante cuáles) tipos de datos en PHP. Los tipos de datos básicos, o comúnmente llamados «escalares», son:



Esquema 2.

La función `gettype()` permite conocer el tipo de una variable. Para preguntar si una variable es de un tipo concreto, podemos usar funciones como `is_bool()`, `is_float()`, `is_integer()`, `is_numeric()` o `is_string()`. Por ejemplo, el siguiente código:

```
$nombre="Elena";  
$numero="1";  
echo gettype($nombre). "<br/>";  
echo gettype($numero). "<br/>";
```

mostraría por pantalla en ambos casos que el tipo es **string**, que es el tipo que se deduce al especificar el contenido entre comillas.

1.4. PHP: constantes, constantes predefinidas y constantes mágicas

Una constante es, por definición, un identificador cuyo valor no puede cambiar durante la ejecución de la aplicación. Aunque en PHP los nombres de las constantes pueden incluir letras minúsculas, por convención se declaran en mayúsculas, siguiendo las mismas reglas aplicadas a los nombres de las variables, pero excluyendo el signo **\$** al principio.

Las constantes se definen mediante la instrucción **define**, a la que se deben suministrar tanto el nombre de la constante como su valor.

Por ejemplo, vamos a definir el valor de la constante pi y la usaremos para calcular el valor de la longitud de una circunferencia.

```
<?php
// variable
$radio=3;
// constante
define("PI",3.14159);
echo "La longitud de la circunferencia con radio $radio es ". (2*PI*$radio);
?>
```

El resultado de ejecutar el *script* desde el navegador será:

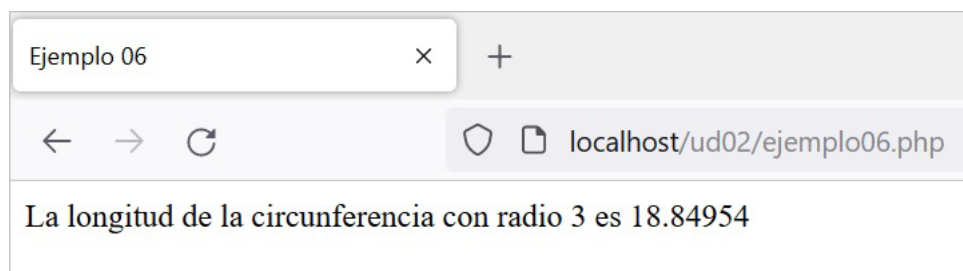


Figura 4.

El lenguaje PHP dispone de un buen número de **constantes predefinidas**. Algunas de ellas son de uso general, mientras que otras están relacionadas con ciertas extensiones del núcleo. Dada la gran cantidad de constantes predefinidas, recomendamos acudir al manual de PHP para ver cuáles tenemos disponibles en cada caso. En el material complementario dispones de varios enlaces de consulta, aunque a lo largo de las unidades iremos viendo algunas de las constantes más usadas.

Así, en el ejemplo anterior no hubiese sido necesario definir la constante pi, puesto que ya existe en PHP con el nombre de **M_PI**.

Las **constantes mágicas** son constantes cuyo valor no está predefinido, sino que se resuelve en tiempo de ejecución. En general, proporcionan información sobre la situación de la aplicación durante la ejecución.

Por ejemplo, la constante mágica `__LINE__` devuelve el número de línea actual del script que está siendo ejecutado. En los enlaces del material complementario puedes consultar otras constantes mágicas disponibles en PHP.

1.5. PHP: variables con nombres generados dinámicamente

PHP permite definir variables con un nombre generado dinámicamente. Para ello podemos usar el contenido de otra variable como nombre. La nueva variable se definiría colocando un signo `$` adicional delante del nombre de la variable definida previamente. Por ejemplo, para dar la bienvenida en castellano, italiano o inglés, podríamos hacer:

```
<?php
// variables con el saludo en diferentes idiomas
$m_en="Hello";
$m_es="Hola";
$m_it="Ciao";
// variable para definir el idioma que se va a usar en este caso
$id idioma="es";
// variable con nombre generado dinámicamente
$bienvenida="m_".$id idioma; // el nombre de la variable en este ejemplo
será m_ca
// usamos la variable generada dinámicamente
echo "¡".$$bienvenida."!";
?>
```

El resultado al ejecutar el *script* será:

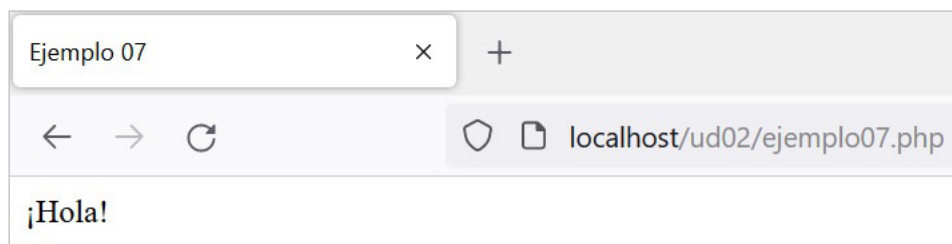
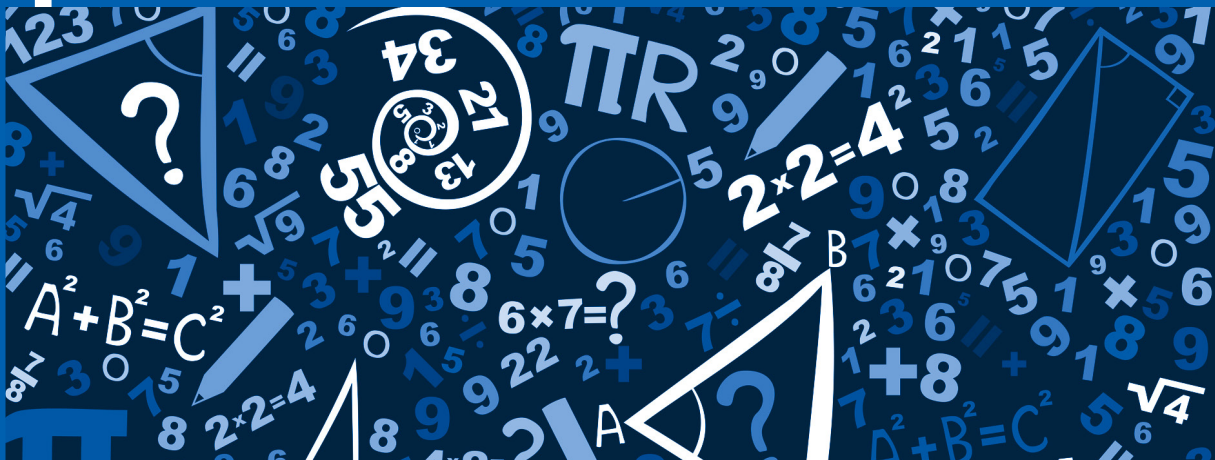


Figura 5.

1

CASO PRÁCTICO

**LONGITUD, SUPERFICIE Y VOLUMEN A PARTIR DE UN RADIO**

Si sabemos el radio de una circunferencia, podemos calcular su longitud, la superficie del círculo asociado y el volumen de la esfera construida a partir de dicho radio. Las fórmulas son:

Longitud de una circunferencia con radio r : $2 * \pi * r$ (π es la constante matemática pi).

Superficie del círculo con radio r : $\pi * r^2$.

Volumen de la esfera con radio r : $4/3 * \pi * r^3$.

Crea un documento PHP en el que defines como variable el radio con un valor cualquiera, siempre que sea mayor que 0. Usando la constante que PHP proporciona para el valor de pi, y aplicando las fórmulas que hemos visto, se mostrarán los valores de la longitud, la superficie y el volumen de los cuerpos generados a partir de dicho radio. Cada uno de esos valores se mostrará con un tamaño de letra distinto aplicando las etiquetas de HTML H1 para la longitud, H2 para la superficie y H3 para el volumen.

SOLUCIÓN

```
<?php
$radio=3; // el radio
$longitud=2*M_PI*$radio; // longitud de la circunferencia
$superficie=M_PI*$radio*$radio; // superficie del círculo
$volumen=4/3*M_PI*$radio*$radio*$radio; // volumen de la esfera
echo "<h1>Longitud de la circunferencia con radio $radio: $longitud</h1>";
echo "<h2>Superficie del círculo con radio $radio: $superficie</h2>";
echo "<h3>Longitud de la circunferencia con radio $radio: $volumen</h3>";
?>
```



Longitud de la circunferencia con radio 3: 18.849555921539

Superficie del círculo con radio 3: 28.274333882308

Longitud de la circunferencia con radio 3: 113.09733552923

02 Cadenas y fechas

Así como los datos numéricos admiten pocas operaciones (aritméticas y de comparación, básicamente), tanto cadenas como fechas permiten muchas más posibilidades, a la hora de mostrarse por pantalla y en su creación y manipulación. Por eso, les dedicamos un apartado específico para estudiar con más detalles la forma de trabajar con estos tipos de variables.



¿SABÍAS QUE...?

En el archivo `DWES_U02_A02.zip` (en la sección de Ejemplos para trabajar y analizar) puedes encontrar todos los ejemplos que se muestran en este apartado.

EJEMPLO

En el archivo `DWES_U02_A02.zip` (en la sección de Ejemplos para trabajar y analizar) puedes encontrar todos los ejemplos que se muestran en este apartado.

2.1. Cadenas

Una cadena o `string` es una variable, constante o literal, formada por caracteres. Estos pueden ser cualquier carácter representable en el juego de caracteres usado en el *script*. Internamente, se tratan como un *array* de bytes, lo que permite realizar ciertas operaciones especiales que no son posibles con otros tipos de datos. A diferencia de otros lenguajes, en PHP una cadena puede tener un tamaño casi ilimitado.

Para crear una variable de tipo `string` debemos asignarle un contenido. Este contenido puede ser resultado de otra operación, o bien un contenido literal. En este caso, se puede crear, principalmente, de dos maneras distintas:

- » **Entrecomillado doble.** Es el más usual. Permite la inclusión de variables y sentencias de escape para caracteres especiales.

- » **Entrecomillado simple.** Es el más sencillo, pero solo permite incluir como caracteres especiales la propia comilla simple y la barra invertida.

A continuación, puedes ver un ejemplo de uso de ambos tipos de entrecomillado, y el resultado obtenido en cada caso.

```

<?php
$variable="contenido";

// las comillas dobles permiten traducir variables y caracteres especiales
echo nl2br("El valor de la variable es $variable.\n El salto de línea se
aplica al usar nl2br().\n");
echo nl2br("El <strong>código HTML</strong> se aplica\n");
echo nl2br("Podemos escribir caracteres como ` y \ \n");
// las comillas simples no permiten la traducción, salvo en casos muy
concretos
echo nl2br('El valor de la variable es $variable.\n El salto de línea no
se aplica.\n');
echo nl2br('El <strong>código HTML</strong> sí que se aplica\n');
echo "<br/>";
echo nl2br('Podemos escribir una comilla simple (\' ) o una barra invertida
(\\) `');
echo nl2br('escapando con el carácter \\\');
?>

```

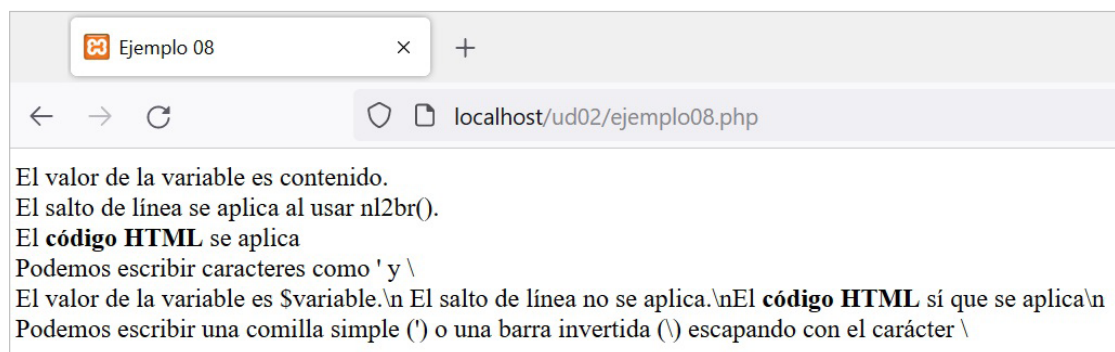


Figura 6.

IMPORTANTE

La función `nl2br()` se usa para convertir los caracteres especiales de salto de línea (`\n`) a saltos de línea reales en el sistema en el que se esté trabajando.

De manera alternativa, se puede generar contenido `string` usando la sintaxis `heredoc` (equivalente a entrecomillado doble) o `nowdoc` (equivalente a entrecomillado simple). La ventaja de `heredoc` sobre el entrecomillado doble es que, al no necesitar las comillas para delimitar la cadena, pueden incluir comillas dobles sin escapar. Con la sintaxis `nowdoc` no se sustituyen las variables ni se escapan los caracteres especiales.

```
<?php
$variable="contenido";
// con heredoc
$texto=<<<EOD
<strong>Heredoc</strong>: el identificador va sin comillas o entre
comillas dobles<br/>
El contenido de la variable "variable" es $variable <br/>
Se usa para textos largos que deben analizarse y sustituirse <br/>
EOD;
echo $texto;
echo "<br/>";
// con nowdoc
$texto=<<<' EOD'
<strong>Nowdoc</strong>: el identificador va entre comillas simples <br/>
El contenido de la variable "variable" es $variable <br/>
Se usa para textos largos literales, sin análisis ni sustituciones <br/>
EOD;
echo $texto;
?>
```

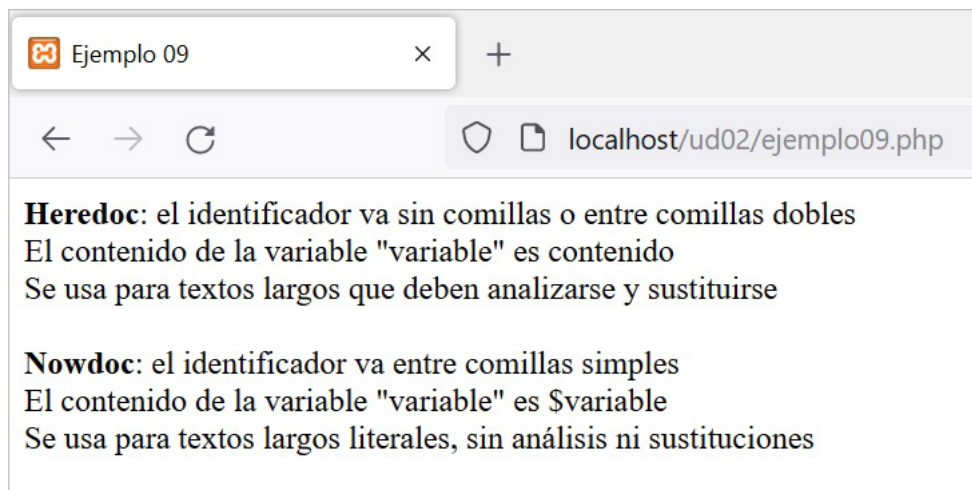
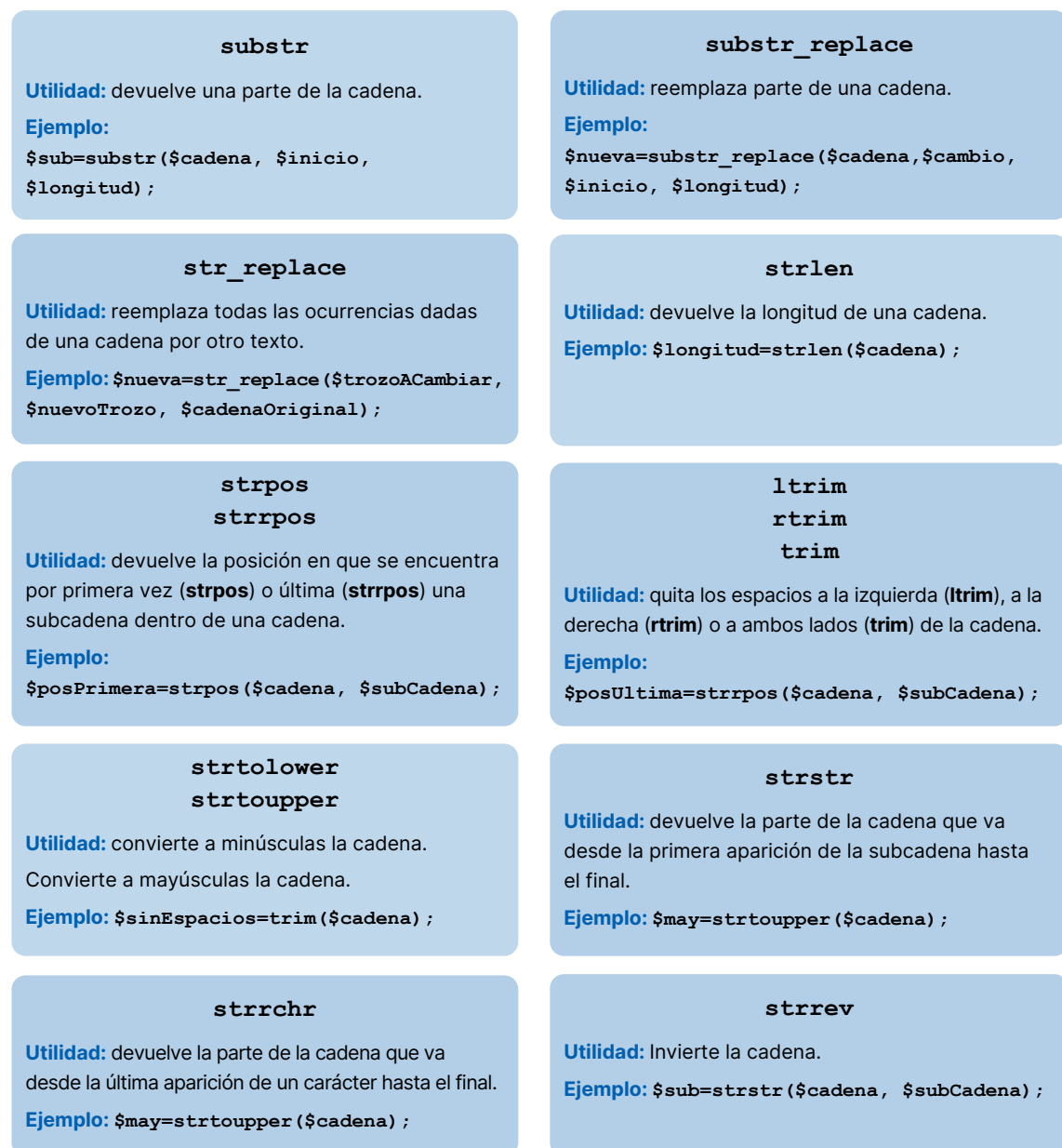


Figura 7.

El lenguaje PHP dispone de multitud de funciones nativas para tratar datos de tipo **string**. Vamos a enumerar las principales, y podrás consultar el resto en los enlaces propuestos.



Esquema 5.

2.2. Fechas

Aunque no existe un tipo explícito para las fechas, sí que tenemos un tipo implícito para las variables en las que guardamos una marca temporal en determinados formatos. Además, PHP ofrece diversas clases para su gestión, así como numerosas funciones para manejar datos de este tipo.

En primer lugar, tenemos la función `date()`, que recibe como primer parámetro el formato requerido para la fecha, y como segundo parámetro (opcional) la fecha que queremos mostrar. Si no le pasamos el segundo parámetro, la función trabaja con la fecha actual del servidor en el que se esté ejecutando el *script*. Esto es importante, porque puede que la zona horaria del servidor no coincida con la del cliente. La función `date()` devuelve como resultado una cadena con la fecha requerida formateada según el patrón especificado.

Para el formato podemos usar los siguientes símbolos:

d	Día del mes (número con ceros iniciales)	w	Semana del año	h	Hora en formato 24 h (2 dígitos con 0 a la izquierda)
j	Día del mes (número sin ceros iniciales)	m	Mes del año (en número, con 0 a la izquierda)	H	Hora en formato 12 h (2 dígitos con 0 a la izquierda)
D	Día de la semana (tres letras, inglés)	n	Mes del año (en número, sin 0 a la izquierda)	g	Hora en formato 12 h (2 dígitos sin 0 a la izquierda)
l	Día de la semana (texto en inglés)	M	Mes del año (tres letras en inglés)	G	Hora en formato 24 h (2 dígitos sin 0 a la izquierda)
N	Día de la semana (en número, lunes = 1)	F	Mes del año (texto en inglés)	i	Minutos (con 0 a la izquierda)
w	Día de la semana (en número, domingo = 1)	y	Año (2 dígitos)	s	Segundos (con 0 a la izquierda)
z	Día del año (0 - 365)	Y	Año (4 dígitos)	a	AM / PM

Así, por ejemplo, el siguiente código

```
<?php
echo "Fecha actual: ";
echo date("d-m-Y")."<br/>";
echo "Fecha y hora actual: ";
echo date("d-m-Y\th:i:s")."<br/>";
echo "Hoy es: ";
echo date("l")." ,".date("d")." de ".date("F")." de ".date("Y")."<br/>";
?>
```

mostrará por pantalla:

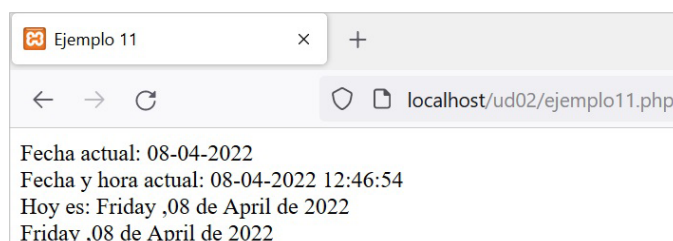


Figura 8.

Para crear una fecha que no sea la del día actual, podemos usar la función `mktime()`, que genera una fecha/hora determinada por el usuario, y pasársela como parámetro a `date()`. La función `mktime()` admite como parámetros, en este orden, la hora, los minutos, los segundos, el mes, el día del mes y el año de la fecha que queremos generar. Así, al ejecutar un *script* con el siguiente código:

```
<?php
echo "Fecha actual: ";
echo date("d-m-Y")."<br/>";
// generamos una fecha nueva con mktime
$fecha=mktime(10,15,35,10,5,2019);
$dia=date("l, d-m-Y",$fecha);
$hora=date("H:i:s",$fecha);
echo "Fecha generada con mktime: $dia $hora <br/>";
?>
```

obtenemos como resultado:

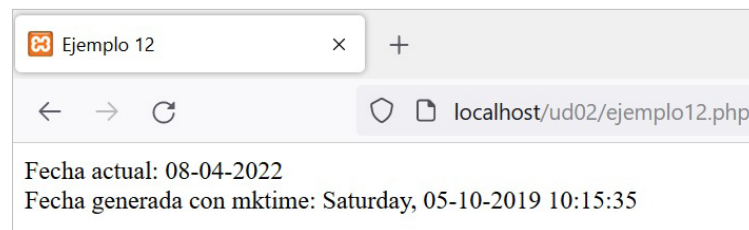


Figura 9.

También disponemos de funciones para calcular la diferencia entre dos fechas, o calcular una fecha a partir de otra, especificando el intervalo de tiempo transcurrido. En este caso, las fechas tienen que ser objetos de la clase `DateTime`. Las podemos crear usando la función `date_create()`. Deben tener formato `mm/dd/YYYY`. Para mostrar la diferencia, debemos usar el método `format()`, pasándole como parámetro el símbolo de la magnitud en que queremos la diferencia entre ambos días. Por ejemplo, calculemos los días que quedan hasta final de año:

```
<?php
// la fecha actual
$hoy=date_create(date("m/d/Y"));
// otra fecha (fin de año)
$fecha=date_create("12/31/2022");
$diferencia=date_diff($hoy,$fecha);
$diasQueFaltan=$diferencia->format("%a");
echo "Faltan $diasQueFaltan días para fin de año";
?>
```

Da como resultado:

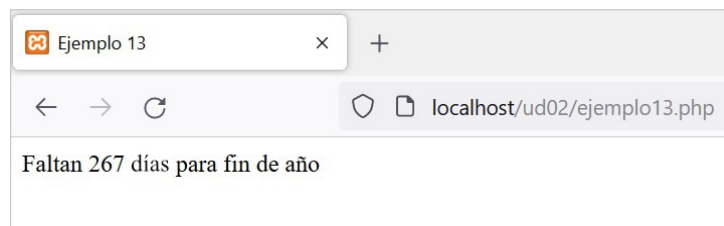


Figura 10.



¿SABÍAS QUE...?

En los enlaces del material adicional puedes encontrar más información sobre el tratamiento de fechas/horas en PHP, incluyendo todo lo necesario para resolver problemas relacionados con las zonas horarias, las clases y los métodos que puedes utilizar y muchas más funciones, aparte de las que hemos visto en este apartado.

03 Estructuras de control

Como en cualquier lenguaje estructurado, en PHP también existen estructuras de control de dos tipos: alternativas (o condicionales), que en función de una o varias condiciones modifican la dirección del flujo del programa; e iterativas (también denominadas popularmente bucles), que permiten que dicho flujo pueda revertirse, de forma que cierta instrucción o grupos de instrucciones se ejecuten varias veces.



¿SABÍAS QUE...?

En el archivo **DWES_U02_A03.zip** (en la sección de Ejemplos para trabajar y analizar) puedes encontrar todos los ejemplos de uso que se muestran en este apartado.

3.1. Estructuras alternativas/condicionales: *if*

La estructura condicional `if` existe en prácticamente todos los lenguajes de programación utilizados hoy en día. Consiste en la evaluación de una o varias condiciones, y en función de si el resultado es verdadero o falso se ejecutan (o no) dos conjuntos distintos de instrucciones. Como mínimo habrá un conjunto de instrucciones que se ejecutarán cuando la evaluación de la condición sea verdadera (bloque `if`). Opcionalmente puede haber otro bloque que se ejecute si la condición es falsa (bloque `else`), e incluso bloques adicionales que se ejecutarán si no se cumple la condición del `if` pero se cumple otra condición que se evalúa en cada bloque (bloques `elseif`).

Resumiendo, la estructura condicional con `if` sigue el siguiente esquema:

IF DE UN SOLO BLOQUE	IF CON DOS BLOQUES	IF MULTICONDICIÓN
<pre>if(condición) { instrucciones; // se ejecutan si la // condición o // condiciones del if se // evalúan a verdadero }</pre>	<pre>if(condición) { instrucciones; // si la condición es // verdadera } else { instrucciones; // si la condición es // falsa }</pre>	<pre>if(condición1) { instrucciones; // si la condición1 es // verdadera } elseif(condición2) { instrucciones; // si la condición1 es // falsa pero la 2 es // verdadera } elseif(condición3) { instrucciones; // si ninguna de las // condiciones anteriores // es verdadera, pero la // condición3 lo es } ... // podemos poner cuantos ... // elseif necesitemos else { instrucciones; // si ninguna de las // condiciones es // verdadera }</pre>

Cuando solo hay dos bloques (`if` y `else`), se puede utilizar también el operador ternario `?`:

```
$resultado = condición ? valor_si_verdadera : valor_si_falsa;
```

Veamos un ejemplo: queremos saber si un número determinado es par o impar, y mostrar un mensaje que lo indique. Comparemos ambas opciones: bloque `if...else` y operador ternario.

```
<?php
$numero=111;
// con if...else
if($numero % 2 == 0) { $mensaje="es par"; }
else { $mensaje="es impar"; }
echo "<h3>Usando un bloque if...else</h3>";
echo "El número $numero $mensaje<br/>";
// con el operador ?
echo "<h3>Usando el operador ternario ?</h3>";
echo "El número $numero \" . ($numero % 2 == 0 ? "es par" : " es impar") . "<br/>";
?>
```

En este caso se aprecia rápidamente que, cuando la condición es sencilla y solo admite dos alternativas, es mucho más rápido y eficiente el uso del operador ternario.

Para ver un ejemplo con tres opciones, vamos a considerar que el 0 no es ni par ni impar.

```
<?php
$numero=0;
// con if...elseif...else
if($numero==0) { $mensaje="es cero"; }
elseif($numero % 2 == 0) { $mensaje="es par"; }
else { $mensaje="es impar"; }
echo "<h3>Usando un bloque if...else</h3>";
echo "El número $numero $mensaje<br/>";
// con el operador ?
echo "<h3>Usando el operador ternario ?</h3>";
echo "El número $numero \" . ($numero==0 ? "es cero" : ($numero % 2 == 0 ?
"es par" : " es impar")) . "<br/>";
?>
```

Como se puede ver, cuando hay más de dos opciones pueden usarse indistintamente bloques `if...elseif...else` o bien operadores ternarios anidados, aunque, cuando las condiciones se complican y las opciones aumentan, entonces `if...elseif...else` proporciona una estructura visualmente más clara.

Hemos de tener en cuenta que, para evaluar ciertas condiciones complejas, podemos usar los operadores lógicos `&&` (and), `||` (or) y `!` (no). Por ejemplo, para saber si un número está entre 1 y 10 hacemos `if($numero >= 1 && $numero <=10)`, y, para preguntar lo contrario, podemos negar la condición con `if(!($numero >= 1 && $numero <= 10))`.

3.2. Estructuras alternativas/condicionales: *switch*

La estructura **switch** es una alternativa bastante interesante respecto a **if...elseif...else** cuando se evalúa una expresión y, en función de su resultado, se ejecuta uno u otro bloque de código. Aunque ambas estructuras son equivalentes, veremos que en ciertos casos **switch** puede ser preferible. Su sintaxis es:

```
switch (expresión) {  
    case valor1:  
        // instrucciones;  
        break;  
    case valor2:  
        // instrucciones;  
        break;  
    ...  
    // opcional  
    default:  
        // instrucciones  
}
```

La expresión se evalúa y proporciona un resultado.

Si el resultado coincide con alguno de los valores, se ejecuta la sección **case** correspondiente.

La instrucción **break** evita que, al acabar de ejecutar una sección **case**, el programa siga con la siguiente

La sección **default** se ejecuta si ninguno de los valores de las secciones **case** coincide con el resultado de la expresión.

Tabla 1.

Al contrario de lo que ocurre en otros lenguajes, al menos hasta tiempos muy recientes, la expresión valorada con **switch** puede dar como resultado una cadena, no solo valores numéricos.

La ventaja del **switch** sobre el **if...elseif** es que resulta más sencillo agrupar casos que se comportan de la misma manera. Para entenderlo, veamos cómo saber cuántos días tiene un mes (como no sabemos el año, supondremos que febrero tiene 28) con un **if** y con un **switch**.

CON if...ELSEIF...ELSE	CON SWITCH
<pre> \$mes=12; if(\$mes==1 \$mes==3 \$mes==5 \$mes==7 \$mes==8 \$mes==10 \$mes==12) { \$mensaje="tiene 31 días"; } elseif(\$mes==4 \$mes==6 \$mes==9 \$mes==11) { \$mensaje="tiene 30 días"; } elseif(\$mes==2) { \$mensaje="tiene 28 días"; } else { \$mensaje="no es un mes válido"; } echo "El mes \$mes \$mensaje
"; </pre>	<pre> switch(\$mes) { case 1:case 3:case 5:case 7:case 8:case 10:case 12: \$mensaje="tiene 31 días"; break; case 4:case 6:case 9:case 11: \$mensaje="tiene 30 días"; break; case 2: \$mensaje="tiene 28 días"; break; default: \$mensaje="no es un mes váli- do"; } echo "El mes \$mes \$mensaje
"; </pre>

Esquema 7.

Con **switch** solo enumeramos los valores que hacen que se ejecute cada bloque sin tener que evaluar la condición cada vez, como ocurre con **if...elseif...else**.

3.3. Estructuras iterativas/repetitivas: *while*

La estructura **while** puede adoptar dos formas. En ambas se evalúa una condición (o varias) y, si el resultado es verdadero, se vuelve a ejecutar el bloque de instrucciones que contiene **while**. En caso de que el resultado sea falso, no se vuelve a ejecutar el contenido del bloque **while** y el flujo de la ejecución sigue su curso.

La sintaxis de **while** puede ser una de las siguientes:

<pre> while(condición) { // instrucciones; } </pre>	<pre> do { // instrucciones; } while(condición); </pre>
---	---

Esquema 8.

La diferencia es que, en el primer caso, la condición se evalúa antes de ejecutar las instrucciones, con lo que, si resulta ser falsa, entonces el bloque interior del **while** no se llegará a ejecutar nunca. En el segundo caso, al evaluarse la condición al final del bucle una vez ejecutadas las instrucciones, estas siempre se ejecutarán al menos una vez.

Lógicamente, dentro del bloque de instrucciones ha de ocurrir algo que cambie la condición de verdadera a falsa, puesto que, de lo contrario, estaríamos ante un bucle infinito que no acabaría nunca. Si antes de ejecutar las instrucciones ya tenemos información para saber si la condición es verdadera o no, es preferible la primera opción (con **while** al principio); si hasta que no hayamos ejecutado las instrucciones no podemos saber si la condición será verdadera o no, entonces deberemos usar la segunda opción (con **while** al final).

Veamos cómo realizar una cuenta desde 1 hasta 10 usando ambas opciones:

<pre>\$inicio=1; \$fin=10; while(\$inicio <= \$fin) { echo \$inicio++."
"; }</pre>	<pre>\$inicio=1; \$fin=10; do { echo \$inicio++."
"; } while(\$inicio <= \$fin);</pre>
---	---

Esquema 9.

En este caso, ambas opciones son equivalentes, y el contenido del bucle es el mismo. Pero ¿qué sucedería si el valor de **\$inicio** fuera mayor que el de **\$fin** antes de empezar el bucle? En el primer caso se detecta que la condición no se cumple, y no se ejecuta el bucle. En el segundo caso se mostraría un valor superior a 10 al menos una vez, puesto que hasta que no se pasa una vez por el bucle no se evalúa la condición.

Resumiendo: si la condición inicial ya se conoce antes de la ejecución del bucle, es mejor la primera opción (**while**), mientras que, si la condición inicial no se conoce hasta que no se ejecuta el bucle, es mejor la segunda (**do...while**).

3.4. Estructuras iterativas/repetitivas: *for*

Aunque en realidad con un bucle **for** se puede hacer prácticamente todo lo que podemos hacer con un bucle **while**, tradicionalmente los bucles **for** se usan para realizar iteraciones sobre una estructura o una lista de valores desde un valor inicial hasta un valor final. Su sintaxis es:

```
for(contador=valor_inicial;condición;incremento) {
// instrucciones;
}
```

El bucle se ejecuta partiendo de un contador con un valor inicial que será incrementando (o decrementando) a cada iteración en función de lo especificado en **incremento**. Cada vez se evaluará si la **condición** se cumple o no, y, en caso de que no se cumpla, entonces el bucle acabará.

Por ejemplo, para contar desde 1 hasta 10 es mucho más claro y eficiente usar un bucle **for**:

```
for($contador=1;$contador<=10;$contador++) {  
    echo "$contador <br/>";  
}
```

Incluso si quisiéramos mostrar solo los números impares, también es más rápido con un **for**, puesto que simplemente deberíamos cambiar el incremento para que vaya de dos en dos:

```
for($contador=1;$contador<=10;$contador+=2) {  
    echo "$contador <br/>";  
}
```

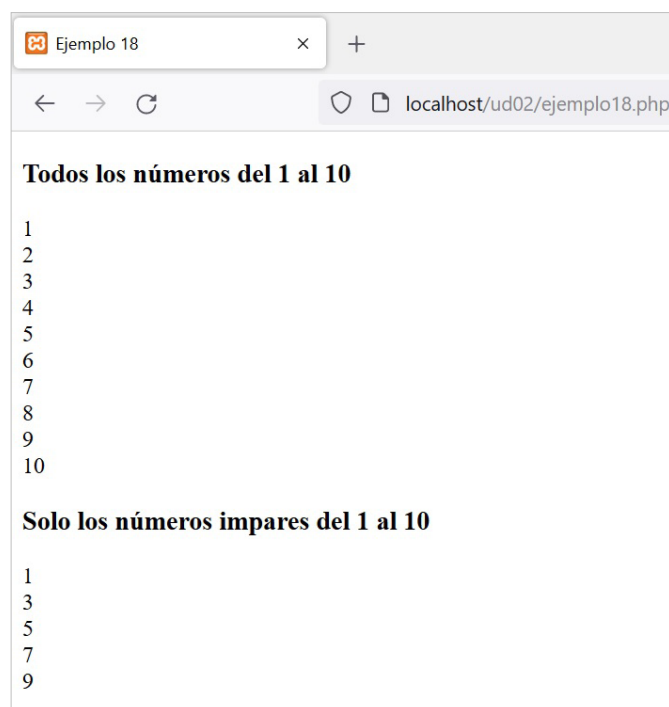


Figura 11.

Como hemos comentado, el bucle funciona tanto con incrementos como con decrementos. Por ejemplo, para contar desde 10 hasta 1:

```
for($contar=10;$contar>=1;$contar--) {echo "$contar <br/>";}
```

El bucle **for** también permite iterar a través de cadenas o fechas. Por ejemplo, vamos a mostrar los próximos diez días partiendo, e incluyendo, la fecha actual. Para ello haremos uso de las funciones de fecha que conocemos, y también de la posibilidad de inicializar más de una variable en la primera sección del bucle y realizar varios incrementos en la sección del incremento.


```
<?php
$hoy=date("d-m-Y");
for($dia=strtotime($hoy), $cont=1;$cont<=10;$dia=strtotime("+1day",$dia),
$cont++) {
    echo date("d-m-Y",$dia)."<br/>";
}
?>
```

Como podemos ver, en la primera parte del `for` inicializamos tanto la variable `$dia` como el contador `$cont`. En la segunda parte del bucle controlamos que no superemos las 10 iteraciones. En la tercera parte del bucle incrementamos en un día el valor de `$dia` y también incrementamos en 1 el valor de `$cont`. El resultado es:

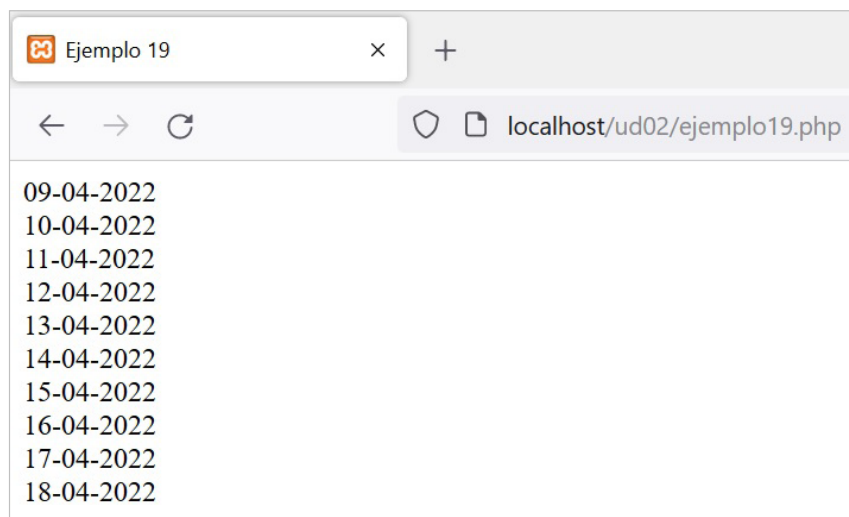


Figura 12.

IMPORTANTE

Dentro de cualquier bucle podemos forzar la salida, sin evaluar la condición, con la instrucción **break**. Con la instrucción **continue** podemos, además, forzar el salto a la siguiente iteración aunque no hayamos completado la actual. Incluso podemos forzar que el programa acabe con la instrucción **exit**. No se recomienda, sin embargo, usar estas instrucciones si hay alternativas usando programación estructurada.

Existe una variación de `for` llamada **foreach** que sirve para repasar valores guardados en una colección o estructura de datos compuesta. En el próximo apartado, cuando estudiemos este tipo de estructuras, veremos cómo funciona este tipo de bucle.

04 Arrays y otros tipos compuestos de datos

Los tipos compuestos de datos en PHP se implementan, básicamente, en forma de **arrays**. Se trata de estructuras muy versátiles, puesto que, a diferencia de otros lenguajes, en PHP se permite que estas estructuras sean dinámicas (su tamaño puede ir variando durante la ejecución del programa) y que los datos contenidos en una única estructura sean de tipos diferentes.



¿SABÍAS QUE...?

En el archivo **DWES_U02_A04.zip** (en la sección de Ejemplos para trabajar y analizar) puedes encontrar los ejemplos que se muestran en este apartado.

4.1. Arrays

Un *array* es una estructura que permite almacenar una colección de datos en una sola variable. En PHP los *arrays* funcionan como una lista de pares clave-valor. Es decir, cada dato guardado en el *array* tiene una clave asociada que permite acceder a este. Es importante saber que, aunque por defecto la clave de una posición es la propia posición (empezando desde 0), las claves pueden ser también alfanuméricas. De hecho, en un mismo array podemos tener posiciones referenciadas con un número, y otras referenciadas con una cadena.

Un array puede crearse de dos formas:

```
$array1 = array(valor1, valor2, valor3...);
```

```
$array2 = [valor1, valor2, valor3...];
```

Como no hemos definido claves, sino solo valores, las posiciones serán 0, 1, 2, etcétera. Por ejemplo, si tenemos un array definido como

```
$array3 = array(10, 5, 8, 14, 25);
```

su representación clave-valor sería:

CLAVE/POSICIÓN:	0	1	2	3	4
VALOR:	10	5	8	14	25

Tabla 2

El tamaño de un array puede ampliarse durante la ejecución del programa. Por ejemplo, si quisiéramos añadir un dato más al array anterior, podríamos hacer:

```
$array3[5]=50; // crea la posición 5 y guarda en ella el valor 50
```

Es más, no necesitamos saber que la última posición es la 4 y la siguiente será la 5:

```
$array3[]=50; // crea la posición siguiente a la última (en este caso, la 5)
```

Gracias a este dinamismo de los arrays, podemos crear uno vacío e ir añadiendo elementos más adelante. Un array vacío se crea haciendo `$nombre=array()`.

Estos arrays con posiciones numeradas correlativamente son muy fáciles de manejar. Por un lado, podemos acceder directamente al contenido de una posición:

```
$valor=$array3[3]; // guardaría el valor 14 en la variable $valor
echo $array3[0]; // mostraría el valor 10
```

Por otro lado, al tratarse de posiciones correlativas, podemos recorrer un array fácilmente con un bucle `for`. Para ello, eso sí, necesitaríamos saber el tamaño del array. Por suerte, disponemos de la función `count()`, que, si le pasamos como parámetro el nombre de un array, nos devuelve la cantidad de elementos que contiene. Así, para mostrar el contenido del array del ejemplo, haríamos:

```
<?php
$array3=array(10,5,8,14,25);
for($posicion=0;$posicion<count($array3);$posicion++) {
    echo "Posición: $posicion Contenido: $array3[$posicion]<br/>";
}
?>
```

Conviene fijarse en que la condición para seguir recorriendo el array es que la posición sea menor que la longitud, y no menor o igual. Eso se debe a que, tal como ocurría con los datos de tipo `string`, al ser la primera posición la 0, la última es igual a la longitud del array menos 1. De hecho, un `string` puede tratarse como si fuera un array de caracteres.

4.2. Arrays asociativos

Aunque en PHP, en realidad, todos los arrays son asociativos, tiende a denominarse así a aquellos cuyas claves se definen independientemente de las posiciones. En estos arrays, la clave puede ser numérica o alfanumérica, indistintamente. Un array con claves alfanuméricas puede usarse, por ejemplo, para encontrar rápidamente un valor a partir de otro con el que se empareja. Por ejemplo, si quisiéramos guardar los teléfonos de varios contactos, podríamos hacer algo como:

```
$telefonos=array("Juan"=>"66611223","Maria"=>"65432101",...);
```

Cuando busquemos el teléfono de Juan, sería tan sencillo encontrarlo como hacer:

```
echo $telefonos["juan"];
```

Accedemos al contenido de la posición «Juan» igual que haríamos con una posición numérica.

Podríamos añadir teléfonos haciendo simplemente:

```
$telefonos["nombre"]="telefono";
```

Se crearía la posición «nombre» con el contenido «telefono».

Debemos tener en cuenta que, si creamos un nuevo elemento sin clave, se le asignará una clave numérica que se corresponderá con el primer número libre empezando desde 0. Es decir, si en el ejemplo anterior hacemos

```
$telefonos[]="nuevo_telefono";
```

el nuevo teléfono se asignará a la clave 0, puesto que es la primera posición numérica libre. El siguiente teléfono que se añadiera sin clave se asignaría a la clave 1, y así sucesivamente. Claro que, si no hay un índice correlativo, ¿cómo podemos ahora recorrer el array? La respuesta es usando el bucle **foreach**, que vimos en un apartado anterior. Este tipo de bucle no necesita que las posiciones sean correlativas, ni empezar desde 0 ni tampoco ir incrementado la posición. Es mucho más sencillo:

```
foreach($array as $valor) {           // para cada $valor del $array
    echo $valor;                       // se muestra el $valor
}
```

Si queremos mostrar también la clave, haremos `foreach($array as $clave=>$valor)`. Por ejemplo, el siguiente código

```
<?php
$telefonos=array("Juan"=>"66611223","Maria"=>"65432101");
$telefonos["Susana"]="6111111";
$telefonos[]="123123123";
foreach($telefonos as $clave=>$valor) {
    echo "<strong>Nombre</strong>: $clave <strong>Teléfono</strong>: $valor<br/>";
}
?>
```

daría como resultado:

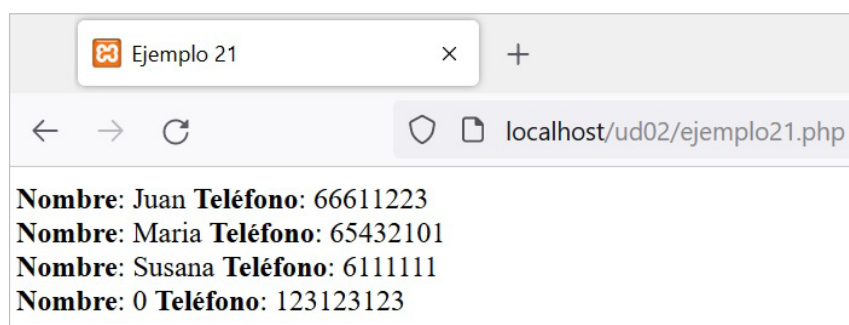


Figura 13.

4.3. Funciones de *arrays*

En PHP disponemos de numerosas funciones para trabajar con arrays. Veamos algunas de ellas:

FUNCIÓN	DESCRIPCIÓN	EJEMPLO
<code>unset()</code>	Elimina un elemento del array o el array entero.	<code>unset(\$array[\$posicion]);</code> <code>unset(\$array);</code>
<code>var_dump()</code>	Muestra el contenido de un array, incluyendo claves, tamaño, valores y tipos.	<code>var_dump(\$array);</code>
<code>print_r()</code>	Muestra el contenido de un array en formato cadena solo con las claves y los valores, sin especificar tipo ni tamaño.	<code>print_r(\$array);</code> <code>\$cadena=print_r(\$array,TRUE);</code>
<code>array_values()</code>	Permite reindexar las posiciones de un array para eliminar huecos producidos por <code>unset()</code> .	<code>\$arrayb=array_values(\$arraya);</code>
<code>array_diff()</code>	Compara un array con otro u otros y devuelve un array con los valores del primero que no aparecen en el resto.	<code>\$dif=array_diff(\$array1,\$array2);</code>
<code>array_fill()</code>	Rellena un array con un valor.	<code>\$array=array_fill(inicio,posiciones,valor);</code>
<code>array_key_exists()</code>	Verifica si una clave existe en el array.	<code>if(array_key_exists(\$clave,\$array))</code>
<code>array_search()</code>	Verifica si un valor existe en el array.	<code>\$clave=array_search(\$valor,\$array);</code>
<code>count()</code>	Cuenta las posiciones de un array.	<code>\$posiciones=count(\$array);</code>
<code>sort</code>	Ordena un array en función de sus datos. Devuelve <i>true</i> si se ha ordenado sin problemas.	<code>sort(\$arraydesordenado);</code>
<code>ksort</code> <code>krsort</code>	Ordena un array en función del valor de sus claves (ksort , de menor a mayor; krsort , de mayor a menor).	<code>ksort(\$arraydesordenado);</code>
<code>asort</code> <code>arsort</code>	Ordena un array en función de su contenido, manteniendo la asociación de los valores ordenados con sus claves originales (asort , de menor a mayor; arsort , de mayor a menor).	<code>asort(\$arraydesordenado);</code>
<code>array_slice</code>	Devuelve un array nuevo con una parte seleccionada de otro array.	<code>\$nuevo=array_slice(\$array,\$inicio,\$posiciones)</code>

Tabla 3.

Otras funciones de gran interés son `implode` y `explode`. Ambas están relacionadas con la conversión de *array* a *string* y viceversa. La función `implode` recibe como parámetros un carácter separador y un array, devolviendo una cadena formada por los elementos del array separados por el carácter especificado. La función `explode` realiza el proceso inverso: convierte una cadena en un array. Para ello, recibe como parámetro un carácter separador y una cadena, dividiendo la cadena en porciones según el carácter especificado y guardando cada porción en una posición del array que se devuelve. Veamos un ejemplo:

```
<?php
$cadena="Esto es una cadena";
echo "<strong>Cadena original:</strong> $cadena <br/>";
$array=explode(" ",$cadena);
echo "<strong>Conversión a array: </strong>";
print_r($array);
echo "<br/><strong>Conversión a cadena con guiones: </strong>";
$cadena2=implode("-", $array);
echo $cadena2;
?>
```

Mostraría lo siguiente:

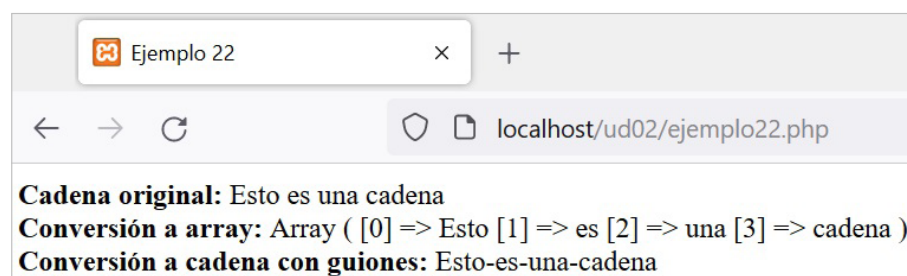


Figura 14.

4.4. Arrays multidimensionales

Los arrays multidimensionales se crean simplemente especificando que el valor de una posición concreta de un array es otro array. Por ejemplo, para crear algo así como una matriz con tres filas y tres columnas como esta:

1	2	3
4	5	6
7	8	9

haríamos:

```
$matriz=array(array(1,2,3),array(4,5,6),array(7,8,9));
```

Para recorrer una matriz con índices numéricos correlativos, como la que acabamos de crear, necesitaremos dos bucles **for** o **foreach** anidados, uno para las filas y otro para las columnas.

```
<?php
$matriz=array(array(1,2,3),array(4,5,6),array(7,8,9));
$numfila=1;
foreach($matriz as $fila) {
    echo "<strong>Fila $numfila: </strong>";
    foreach($fila as $valor) {
        echo $valor." ";
    }
    $numfila++;
    echo "<br/>";
}
?>
```

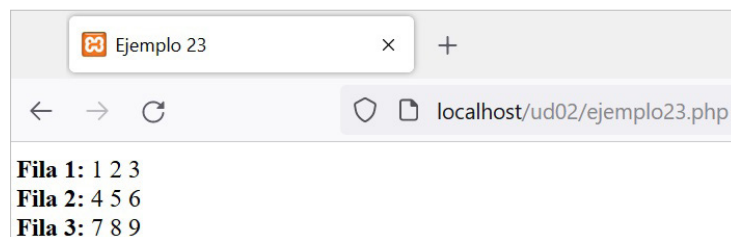


Figura 15.

Lo mismo haríamos si las claves fueran alfanuméricas, salvo que las filas ya no estarían numeradas, sino que tendrían su propia clave. Por ejemplo, supongamos que queremos guardar en un array el nombre, el teléfono y el correo electrónico de algunos contactos. Una posible solución sería que cada fila fuera un contacto y tuviese como clave el nombre. El resto de columnas serían el teléfono y el correo electrónico. Veamos un ejemplo:

```
<?php
$contactos=array("Juan"=>array("Tfno"=>"111111","Email"=>"juan@gmail.com"),
    "María"=>array("Tfno"=>"222222","Email"=>"maria@gmail.com"),
    "Elena"=>array("Tfno"=>"33333","Email"=>"elena@hotmail.com"));
foreach($contactos as $clave1=>$contacto) {
    echo "<br/><strong>Nombre contacto: $clave1: </strong><br/>";
    foreach($contacto as $clave2=>$valor) {
        echo " <strong>$clave2: </strong>$valor";
    }
}
?>
```

Darí­a como resultado:

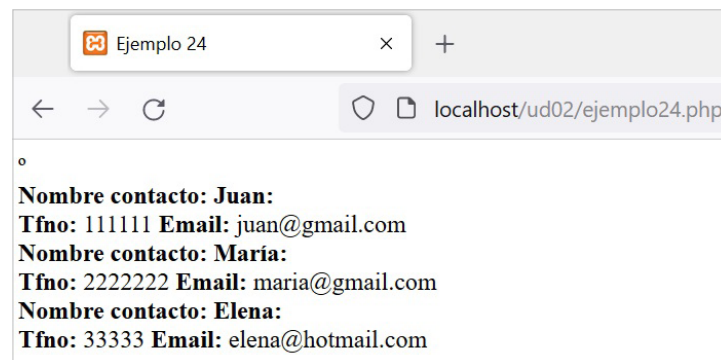


Figura 16.

Las posibilidades de trabajar con arrays asociativos, datos de diferentes tipos y tamaños variables son casi infinitas. Sin embargo, más adelante veremos que, para manejar estos tipos de datos estructurados (por ejemplo, la información de los contactos, las características de un producto, los datos personales de un cliente, etcétera), es mejor usar otra estructura compuesta: las **clases**.

También existe la posibilidad de usar **colecciones**, aunque en general se utilizan en combinación con clases, para trabajar con información obtenida de bases de datos en cualquier formato. Volveremos sobre este tipo de estructuras más adelante.

2 CASO PRÁCTICO



CONTAMOS LOS ELEMENTOS DE UN ARRAY

Disponemos de un *array* con una serie de números, de los cuales algunos de ellos están repetidos. Se pide escribir un *script* en **PHP** que muestre las veces que aparece cada uno de los números en el *array*. Dado, por ejemplo, un *array* con los valores (1,10,5,1,5,5,8), el programa debería mostrar que el número 1 aparece 2 veces, el número 10 aparece una vez, el número 5 aparece 3 veces y el número 8 aparece una vez. Intenta que cuando el número aparece una vez se muestre la palabra «vez», mientras que, en los números que aparezcan más de una vez, se muestre la palabra «veces».

SOLUCIÓN

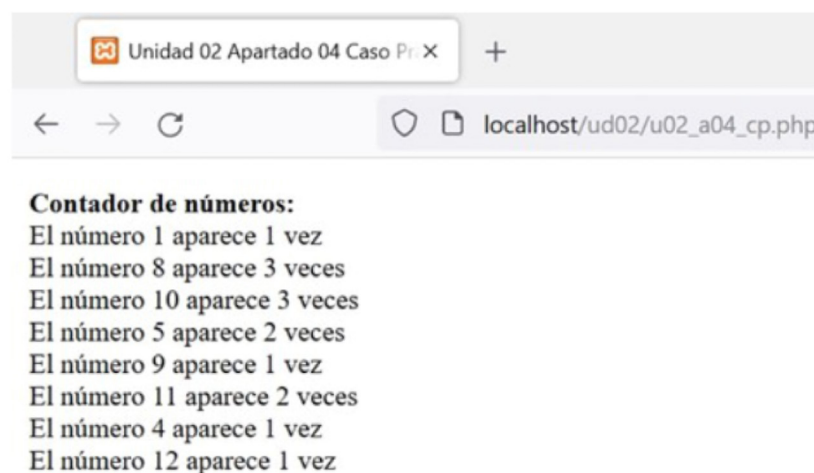
La mejor solución para contar las apariciones de cada elemento en un *array* es crear otro *array* que funcione como contador. Este segundo *array* tendrá como claves los valores del primero, y como valores las veces que aparece el elemento correspondiente en el primer *array*. El código sería:

```
<?php
$array=array(1,8,10,5,8,9,10,11,4,
,8,5,10,11,12);
echo "<strong>Array original: </strong>";
print_r($array);
$arrayContador=array();
foreach($array as $valor) {

    if(array_key_exists($valor,$arrayContador)) {

        $arrayContador[$valor]=$arrayContador[$valor]+1;
    }
    else {
        $arrayContador[$valor]=1;
    }
}
echo "<br/><strong>Contador de números: </strong>";
foreach($arrayContador as $numero=>$veces) {
    echo "<br/>El número $numero aparece $veces ". ($veces==1 ? "vez" : "veces") ;
}
```

Con los estilos añadidos en la cabecera del documento, el resultado se vería así:



Puedes encontrar la solución en el archivo `DWES_U02_CP_A04_SOL.zip`.

05 Funciones. *Include y require*

El concepto de función nos debería ser familiar, dado que en el módulo de Programación de primer curso ya se ha estudiado a fondo. En este apartado veremos cómo crear y utilizar funciones en PHP, así como sus principales particularidades.

EJEMPLO

En el archivo `DWES_U02_A05.zip` (en la sección de Ejemplos para trabajar y analizar) puedes encontrar los ejemplos que se muestran en este apartado.

5.1. Funciones en PHP

Una función es una parte del código de un programa que se delimita, recibe un identificador y solo se ejecuta cuando este es llamado usando dicho identificador. Como en todos los lenguajes de programación, opcionalmente las funciones en PHP pueden recibir parámetros y devolver valores. Aunque una función puede devolver como mucho una variable, dicha variable puede ser una estructura compuesta, que a su vez contenga muchos otros valores. De esa forma, podemos hacer que una función devuelva más de un valor.

Como hemos comentado, las funciones reciben un nombre identificativo, por el que serán llamadas posteriormente. Dicho nombre irá precedido por la palabra reservada **function** y seguido por los nombres de los parámetros entre paréntesis (el paréntesis es obligado, aunque no haya parámetros). El cuerpo de la función está delimitado por llaves `{...}` y al final no se pone punto y coma. Los nombres posibles para las funciones vienen determinados por las mismas reglas vistas para las variables. En este caso, podemos usar mayúsculas o minúsculas indistintamente, aunque es recomendable usar siempre el mismo formato de nombre para referirse a la misma función.

```
<?php
function bienvenida() { echo "Hola, mundo<br/>"; }
bienvenida(); // funciona
Bienvenida(); // también funciona
?>
```

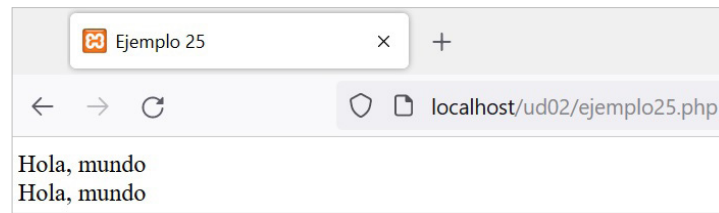


Figura 17.

Una función puede recibir uno o varios parámetros como argumentos. Los nombres de las variables que recibirán los valores pasados como parámetros se incluyen dentro del paréntesis, separados por comas, y posteriormente podrán ser usados en el cuerpo de la función. Al llamar a la función, le pasaremos los valores concretos (directamente o mediante variables) que queremos asignar a los parámetros.

Veamos un ejemplo: modifiquemos nuestra función de bienvenida para que reciba como parámetro el idioma en el que queremos mostrar el mensaje:

```
<?php
function bienvenida($idioma) {
    if($idioma=="es") { echo "Hola, mundo<br/>"; }
    elseif($idioma=="en") { echo "Hello, world<br/>"; }
    else { echo "Lo siento, no reconozco el idioma<br/>"; }
}
bienvenida("es");
bienvenida("en");
?>
```

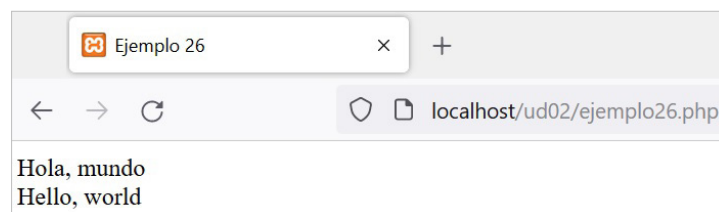


Figura 18.

Opcionalmente las funciones pueden devolver un resultado. Dicho resultado, en forma de una variable o un valor fijo, se devuelve mediante la instrucción **return** y será recibida y tratada en la llamada a la función. Veamos cómo quedaría nuestra función de bienvenida devolviendo el mensaje, en lugar de mostrarlo por pantalla:

```
<?php
function bienvenida($idioma) {
    if($idioma=="es") { $mensaje="Hola, mundo<br/>"; }
    elseif($idioma=="en") { $mensaje="Hello, world<br/>"; }
    else { $mensaje="Lo siento, no reconozco el idioma<br/>"; }
    return $mensaje;
}
```

```

$mensajeRecibido=bienvenida("es"); // guarda el resultado
echo $mensajeRecibido; // y luego trabajamos con él
echo bienvenida("en"); // también puede incluirse directamente
                        // la llamada a la función en el echo

?>

```

El resultado por pantalla será el mismo que acabamos de ver en el anterior ejemplo.

Como se ve en el código, PHP no obliga a especificar los tipos de los parámetros ni tampoco el del resultado devuelto, a diferencia de lo que ocurre con otros lenguajes. Sin embargo, sí que es posible hacerlo y le da más robustez al código. El tipo de los parámetros puede introducirse desde la versión 5 de PHP, mientras que el tipo del valor devuelto se puede especificar a partir de la versión 7.

```
function bienvenida(string $idioma): string {
```

Cuando declaramos un parámetro en una función, le podemos dar un valor por defecto. Así, en el caso de que dicho parámetro no se pase en la llamada, adquirirá dicho valor. Por ejemplo:

```

<?php
function bienvenida(string $sujeto=" a todos"): string {
    return "Hola $sujeto<br/>";
}

echo bienvenida();           // mostrará "Hola a todos"
echo bienvenida("Jaime");    // mostrará "Hola Jaime"

?>

```

De esa forma, podemos tener más flexibilidad a la hora de llamar a la función, pudiendo pasar o no el parámetro en función de si queremos o no usar un valor diferente al propuesto por defecto. Sin embargo, esto solo nos da dos posibilidades: pasar el parámetro o no pasarlo. Pero... ¿y si necesitamos que una función pueda recibir un número diferente de parámetros cada vez que es llamada? PHP está preparado para ello con dos funciones:

FUNCIÓN	UTILIDAD
<code>func_num_args()</code>	Utilizada dentro de una función, devuelve la cantidad de parámetros que recibe cuando es llamada.
<code>func_get_arg()</code>	Devuelve la lista de parámetros recibidos en forma de un array con índice numérico.

Esquema 11.

La primera función nos permite saber cuántos parámetros se han pasado, y la segunda nos los proporciona en un array que podemos recorrer con un bucle `for`. Veamos, como ejemplo, una función que devuelve la suma de cualquier cantidad de números que le pasemos:

```
<?php
function sumaVariable() {
    $total=0;
    for($i=0;$i<func_num_args();$i++) {
        $total=$total+func_get_arg($i);
    }
    return $total;
}
echo sumaVariable(1,2,3)."<br/>";
echo sumaVariable(1,10)."<br/>";
echo sumaVariable(5,8,1,10,11,8,3)."<br/>";
?>
```

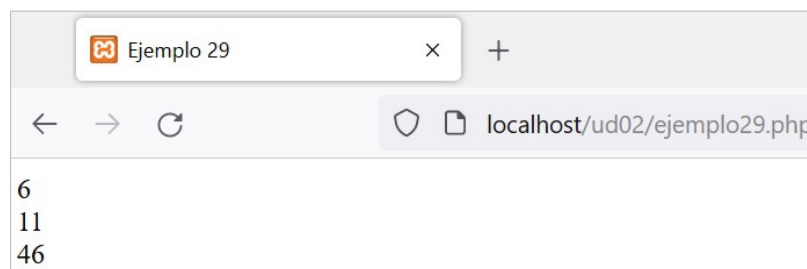


Figura 19.

Otra opción es usar el *token ...* para especificar que una función admite una lista variable de parámetros. En este caso, se especifica el nombre del *array* donde queremos que se guarden. En el ejemplo anterior, la función `sumaVariable` quedaría así:

```
function sumaVariable(...$lista) {
    $total=0;
    foreach($lista as $parametro) {
        $total=$total+$parametro;
    }
    return $total;
}
```

El ámbito de las variables declaradas dentro de la función es siempre local, lo que quiere decir que su valor se pierde una vez salimos de la función. Tampoco podemos acceder a variables externas y modificar su contenido dentro de la función, salvo que se dichas variables se pasen como parámetros por referencia, o que se declaren como globales.

Para pasar una variable como parámetro por referencia a la función, se declara el parámetro precedido por el símbolo `&`. De esa forma, cualquier cambio que se le haga dentro de la función a la variable pasada por referencia permanecerá una vez que la función acabe. Veamos un ejemplo:

```
<?php
function concatena1($cad1, $cad2) { $cad1=$cad1.$cad2; }
function concatena2(&$cad1, $cad2) { $cad1=$cad1.$cad2; }
$cadena1="Hola";
$cadena2=", mundo";
concatena1($cadena1, $cadena2);
echo "<strong>Pasando la cadena1 por valor:</strong> ";
echo $cadena1."<br/>";
concatena2($cadena1, $cadena2);
echo "<strong>Pasando la cadena1 por referencia:</strong> ";
echo $cadena1."<br/>";
?>
```

Las funciones `concatena1` y `concatena2` hacen lo mismo: reciben como parámetro dos cadenas y concatenan la segunda a la primera. Sin embargo, la primera recibe la cadena original por valor, de forma que los cambios que se le hacen en la función se pierden una vez acaba. Por el contrario, la segunda función recibe la cadena original por referencia, de forma que trabaja sobre la misma cadena original declarada en el programa principal y, por tanto, los cambios hechos dentro de la función permanecen después de salir de ella.

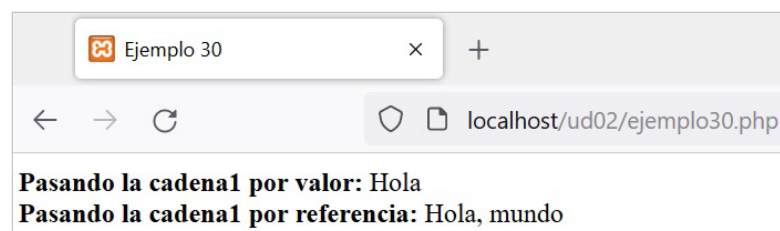


Figura 20.

Otra forma de trabajar con variables externas a la función y poder manipular su contenido es redefinirlas, dentro de la función, como globales. Veamos el mismo ejemplo anterior usando una variable global en lugar de un parámetro por referencia:

```
<?php
function concatena3($cad2) {
    global $cadena1;
    $cadena1=$cadena1.$cad2;
}
$cadena1="Hola";
$cadena2=", mundo";
concatena3($cadena2);
echo "<strong>Redefiniendo cadena1 como global:</strong> ";
echo $cadena1."<br/>";
?>
```

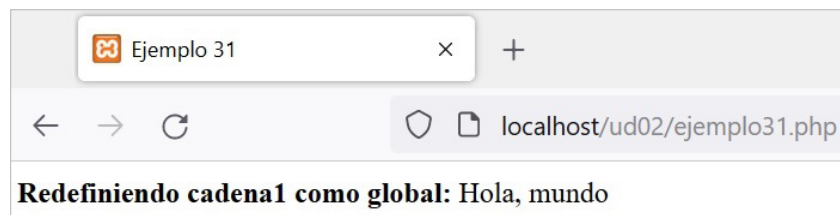


Figura 21.

Una alternativa a la redefinición de la variable externa como global es acceder directamente al array `$GLOBALS` con el índice del nombre de la variable deseada.

```
function concatena4($cad2) {
    $GLOBALS["cadena1"]=$GLOBALS["cadena1"].$cad2;
}
$cadena1="Hola";
$cadena2=", mundo";
concatena4($cadena2);
echo "<strong>Accediendo al vector GLOBALS:</strong> ";
echo $cadena1."<br/>";
?>
```

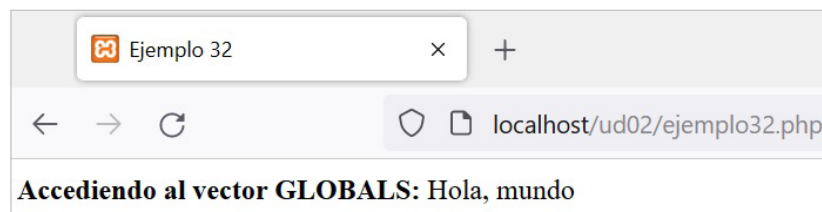


Figura 22.

Otra interesante característica de PHP es que permite guardar el nombre de una función en una variable y llamarla posteriormente usando dicha variable. De esa forma la misma llamada puede hacer referencia a diversas funciones. Esto es útil para evitar múltiples llamadas a funciones en casos en los que deba escogerse una. Por ejemplo, si disponemos de las diferentes funciones de concatenación que hemos visto en los ejemplos, y queremos llamar a una de ellas en función de una opción, podríamos hacer:

```
$opcion=3;
$funcion="concatena".$opcion;
$funcion();
```

En lugar de algo como

```
if($opcion==1) { concatena1(); }
elseif($opcion==2) { concatena2(); }
elseif($opcion==3) { concatena3(); }
...
```

En el documento de enlaces dispones de acceso a más información sobre todas las posibilidades que ofrece la creación y uso de funciones en PHP.

5.2. Include

En ocasiones, el código de un documento PHP se repite exactamente igual en otro documento PHP. En esos casos puede ser más sencillo copiar y pegar que crear una función que realmente no tiene más utilidad que evitar la duplicidad del código. Tenemos, sin embargo, una alternativa que nos evita tener que copiar y pegar, así como crear funciones no requeridas por la aplicación. La alternativa es la cláusula **include**.

La cláusula **include** copia y pega por nosotros, pero en tiempo de ejecución. Es decir, si tenemos un código que se repite en varias páginas, podemos almacenarlo en una página nueva creada exclusivamente para dicho código, y después incluir esa página con **include** en aquellos sitios donde sea necesario.

1

Supongamos que tenemos una serie de funciones creadas por nosotros mismos que necesitamos usar en diferentes documentos PHP. En principio deberíamos tener todas esas funciones repetidas en cada documento. Con **include**, sin embargo, no es necesario.

Creamos un documento que llamamos, por ejemplo, **funciones.php**; copiamos allí todas las funciones que queremos usar en diferentes sitios, y posteriormente, en cada página donde queramos usar alguna de las funciones, haremos:

```
include "funciones.php";
```

2

De esa forma, el código de las funciones está en un único documento, lo que hace más sencillo su mantenimiento, pero puede usarse en otros documentos como si realmente estuviese copiado y pegado en ellos. Suele usarse **include**, además de para reutilizar funciones en diferentes documentos, para incluir cabeceras o pies de página que se repiten de unos documentos a otros.

Supongamos que tenemos una cabecera como esta, que queremos incluir en todos nuestros documentos:

```
<h1 style="color:blue;">DWES: Desarrollo Web en Entorno Servidor</h1>
<h2 style="color:red;">Material para el alumnado</h2>
```

3

En lugar de copiar y pegar el código en cada documento, con el problema de que si posteriormente queremos añadir o modificar algo deberemos hacerlo en todos los documentos, podemos incluir nuestras dos líneas en un fichero llamado, por ejemplo, **cabecera.php**. Así, si ahora en uno cualquiera de nuestros documentos incluimos dicho fichero con **include**, será como si nuestras dos líneas de cabecera estuviesen copiadas y pegadas. Así, el código

```
<?php
include "cabecera.php";
echo "Hola, mundo";
?>
```


daría como resultado:

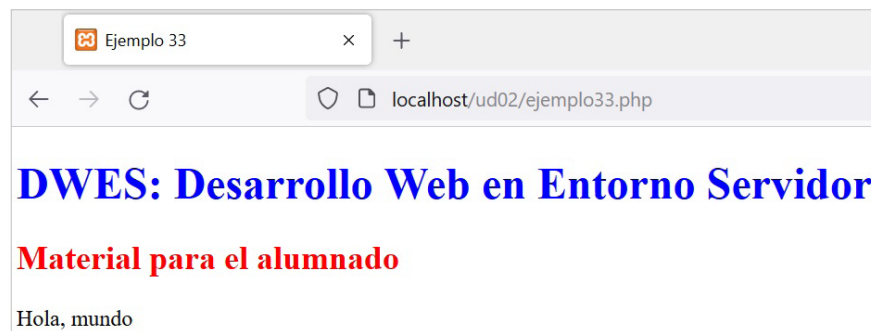


Figura 23.

5.3. Require

La cláusula **require** funciona exactamente igual que **include**, con una diferencia sutil: mientras que **include** lanza un aviso si el fichero referenciado no se encuentra, **require** lanza un error y el programa se detiene. Es decir, con **include** nos podemos permitir que el fichero no exista (aunque consideramos que no sería una buena práctica), pero con **require** es obligatorio que el fichero referenciado exista.

5.4. Include_once y require_once

Las cláusulas **include_once** y **require_once** funcionan exactamente igual que **include** y **require**, respectivamente. La diferencia al añadir el sufijo **_once** es que se detecta si el fichero ya ha sido incluido en algún momento anterior de la ejecución, y, si es así, no se vuelve a incluir. Es interesante usar **include_once** y **require_once** en programas de gran tamaño, donde se necesitan multitud de inclusiones de unos ficheros en otros, ya que al hacerlo así evitamos recargas innecesarias y agilizamos la ejecución de la aplicación.



VÍDEO LAB

Para consultar el siguiente vídeo sobre gestión sala de cine, escanea el código QR o [pulsa aquí](#).

UAX FP